Khoa học Dữ liệu

Notebook: Phân cụm cho dữ liệu địa chất
Năm học: 2025-2026

In [1]:
```
# Import thư viện
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

1. Phân cụm theo cấp bậc – Hierarchical clustering

Ví dụ: Cho các điểm A (1, 1), B(2, 3), C(3, 5), D(4, 5), E(6, 6), F(7, 5) tính ma trận khoảng cách, vẽ dendrogram và gom nhóm chúng.

Các bước để khởi tạo ma trận khoảng cách và vẽ dendrogram thông qua thư viện `spicy` với hàm `scipy.cluster.hierarchy`. Đầu tiên cần import thư viện

In [2]:
```
import scipy.cluster.hierarchy as shc
```

Tiếp theo, khởi tạo bộ dữ liệu X để tính toán

In [3]:
```
X = np.array([
    [1, 1],
    [2, 3],
    [3, 5],
    [4, 5],
    [6, 6],
    [7, 5]
])
```

In [4]:
```
X
```

Out[4]:
```
array([[1, 1],
       [2, 3],
       [3, 5],
       [4, 5],
       [6, 6],
       [7, 5]])
```

In [5]:
```
Z_single = shc.linkage(X, method='single', metric='euclidean')
Z_single
```

Out[5]:
```
array([[2.        , 3.        , 1.        , 2.        ],
       [4.        , 5.        , 1.41421356, 2.        ],
       [0.        , 1.        , 2.23606798, 2.        ],
       [6.        , 8.        , 2.23606798, 4.        ],
       [7.        , 9.        , 2.23606798, 6.        ]])
```

Linkage matrix (Z) lưu lại toàn bộ lịch sử gộp cụm; mỗi hàng biểu diễn việc gộp hai cụm tại một khoảng cách xác định, đồng thời cho biết số điểm trong cụm mới. Ma trận Z sẽ có kích thước là (n-1) * 4 với n là số điểm của bộ dữ liệu.

Như ma trận Z trả về ở phía trên có ý nghĩa như sau:

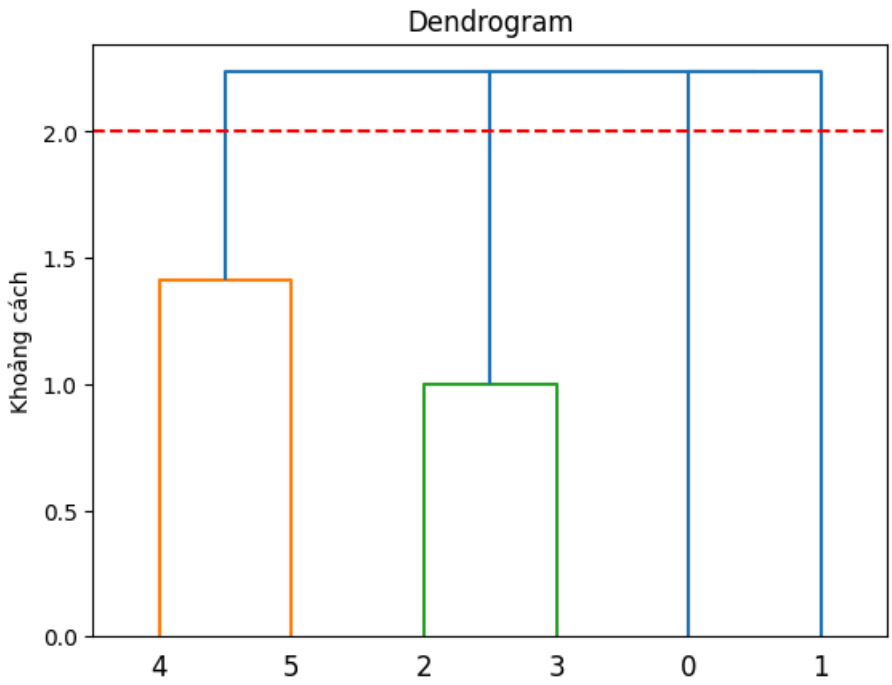
Cột	Ý nghĩa
cột thứ nhất	Chỉ số cụm thứ nhất được gộp
cột thứ hai	Chỉ số cụm thứ hai được gộp
cột thứ ba	Khoảng cách giữa hai cụm khi gộp
cột thứ tư	Số điểm trong cụm mới

Từ đó, có thể hiểu ma trận Z với phần tử đầu tiên là:

- Cụm thứ nhất được khởi tạo với điểm 2 và điểm 3 với khoảng cách là giữa chúng là 1 => Cụm mới tạo có 2 điểm dữ liệu và được gán ID = 6.
- Cụm thứ hai được khởi tạo với điểm 4 và điểm 5 với khoảng cách là giữa chúng là 1.4142 => Cụm mới tạo có 2 điểm dữ liệu và được gán ID = 7.
- Cụm thứ ba được khởi tạo với điểm 0 và điểm 1 với khoảng cách là giữa chúng là 2.236 => Cụm mới tạo có 2 điểm dữ liệu và được gán ID = 8
- Cụm thứ tư được khởi tạo với điểm 6 và điểm 7 với khoảng cách là giữa chúng là 3.1 => Cụm mới tạo có 2 điểm dữ liệu và được gán ID = 9.
- Cụm thứ năm được khởi tạo với điểm 8 và điểm 9 với khoảng cách là giữa chúng là 4.9 => Cụm mới tạo có 2 điểm dữ liệu và được gán ID = 10.

Và để dendrogram, cần sử dụng hàm `dendrogram` để trực quan.

In [6]:
```
shc.dendrogram(Z_single)

plt.axhline(y = 2, color='r', linestyle='--')

plt.ylabel("Khoảng cách")
plt.title("Dendrogram")
plt.show()
```



Có thể thấy với phương pháp **Single** thì **Dendrogram** chia được 4 cụm với khoảng cách tối đa giữa 2 nhóm là 5 đơn vị. Theo đó, nếu ta cắt cây tại khoảng cách (chiều cao) là 2, ta thu được 2 cụm: (4, 5), (2, 3), (0) và (1) và ta đang chấp nhận sự khác biệt tối đa giữa các cụm là 2 đơn vị. Ngược lại, nếu ta khắt khe hơn, chỉ cho phép sự khác biệt tối đa giữa các cụm là 0.5 đơn vị, thì ta thu được 6 cụm - chính là 6 điểm dữ liệu ban đầu.

Ngoài ra, để trực quan các điểm theo số cụm mà các điểm đó thuộc về bằng biểu đồ phân tán, chúng ta cần sử dụng thư viện `sklearn.cluster` với hàm `AgglomerativeClustering`.

In [7]:
```
# Thêm thư viện
from sklearn.cluster import AgglomerativeClustering
```

Vì dữ liệu đầu vào là một mảng gồm 6 phần tử, ta cần chuyển đổi dạng dữ liệu thành `DataFrame` **X_df_single**

In [8]:
```
# Chuyển thành định dạng DataFrame
X_df_single = pd.DataFrame(X, columns=['X1', 'X2'])
```

Sau đó, chúng ta gọi mô hình với hàm **AgglomerativeClustering** đã được thêm vào ở phía trên.

Lưu ý:: Với các tham số như số lượng cụm muốn tạo thành (n_clusters); phương pháp để xác định khoảng cách giữa 2 cụm (linkage) và công thức tính khoảng cách (metric) cần đảm bảo giống với lựa chọn của dendrogram.

In [9]:
```
# Gọi hàm để phân loại
model_single = AgglomerativeClustering(
    n_clusters=4,        # Số lượng nhóm
    linkage='single',    # Cách tính khoảng cách tâm cụm
```

```
    metric='euclidean'  # Cách tính khoảng cách giữa các điểm
)
```

Sau đó, gọi mô hình `model_single` đã được định nghĩa phía trên và áp dụng vào DataFrame **X_df_single**, đồng thời tạo thêm cột `Cluster` để lưu giá trị cụm cho từng điểm dữ liệu. Kết quả thu được sẽ là một DataFrame gồm các điểm dữ liệu và ID cụm của điểm chúng.

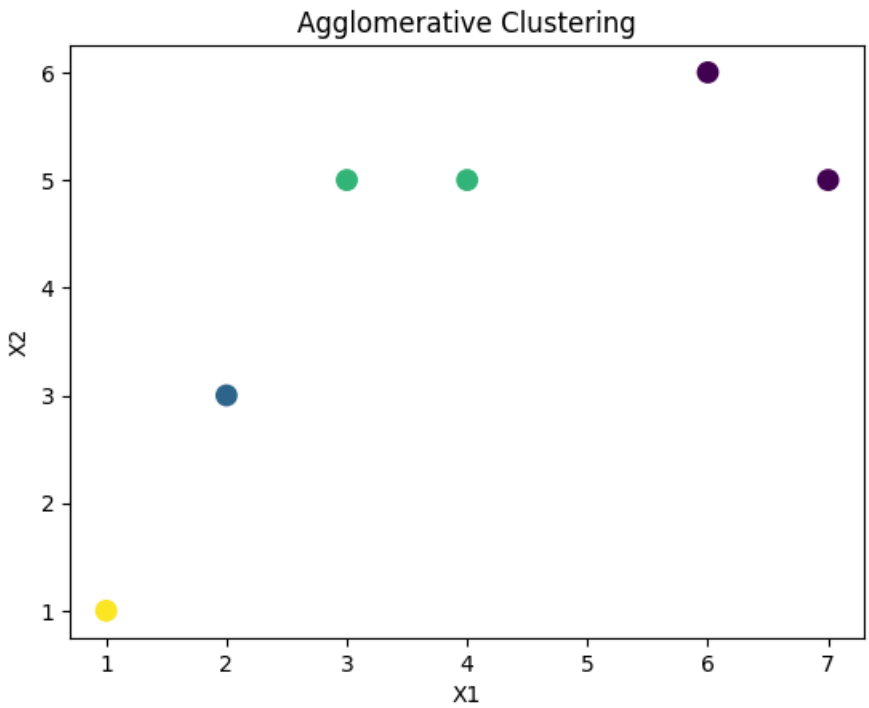
In [10]:
```
X_df_single['cluster'] = model_single.fit_predict(X_df_single)
X_df_single
```

Out[10]:

	X1	X2	cluster
0	1	1	3
1	2	3	1
2	3	5	2
3	4	5	2
4	6	6	0
5	7	5	0

Cuối cùng, vẽ các điểm đó lên biểu đồ phân tán với các cụm nhóm của dữ liệu.

In [11]:
```
plt.scatter(
    X_df_single['X1'],
    X_df_single['X2'],
    c=X_df_single['cluster'],
    s=80
)

plt.xlabel('X1')
plt.ylabel('X2')
plt.title('Agglomerative Clustering')
plt.show()
```

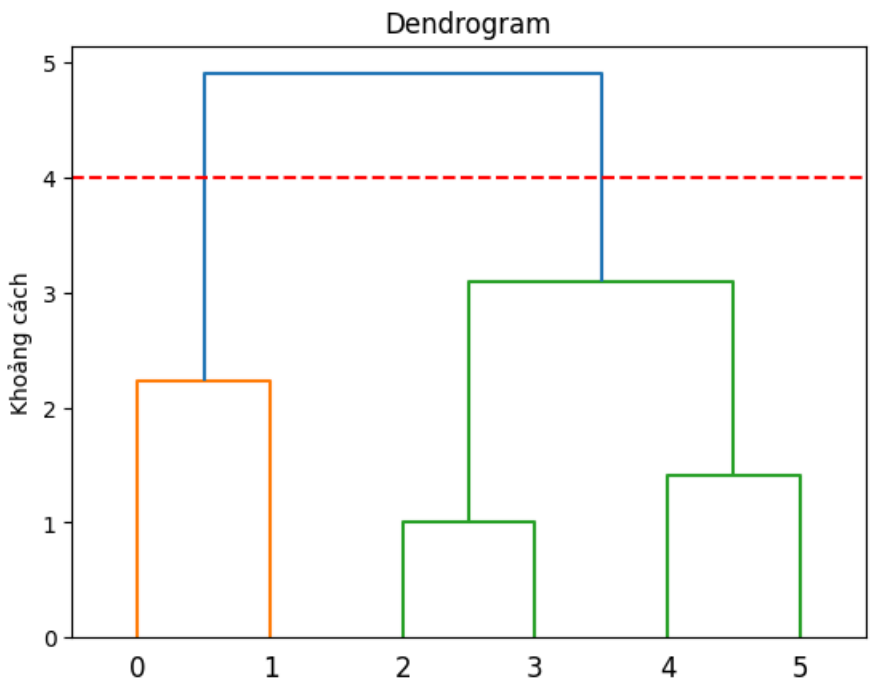


Ngoài ra, có thể sử dụng các phương pháp như **average** và **complete** để xác định Dendrogram và phân cụm cho dữ liệu với việc thay đổi tham số `linkage`.

Như ví dụ phía dưới sẽ vẽ Dendrogram và biểu đồ phân cụm với phương pháp **average**

In [12]:
```
Z_average = shc.linkage(X, method='average', metric='euclidean')
shc.dendrogram(Z_average)

plt.axhline(y = 4, color='r', linestyle='--')

plt.ylabel("Khoảng cách")
```

```
plt.title("Dendrogram")
plt.show()
```



In [13]:

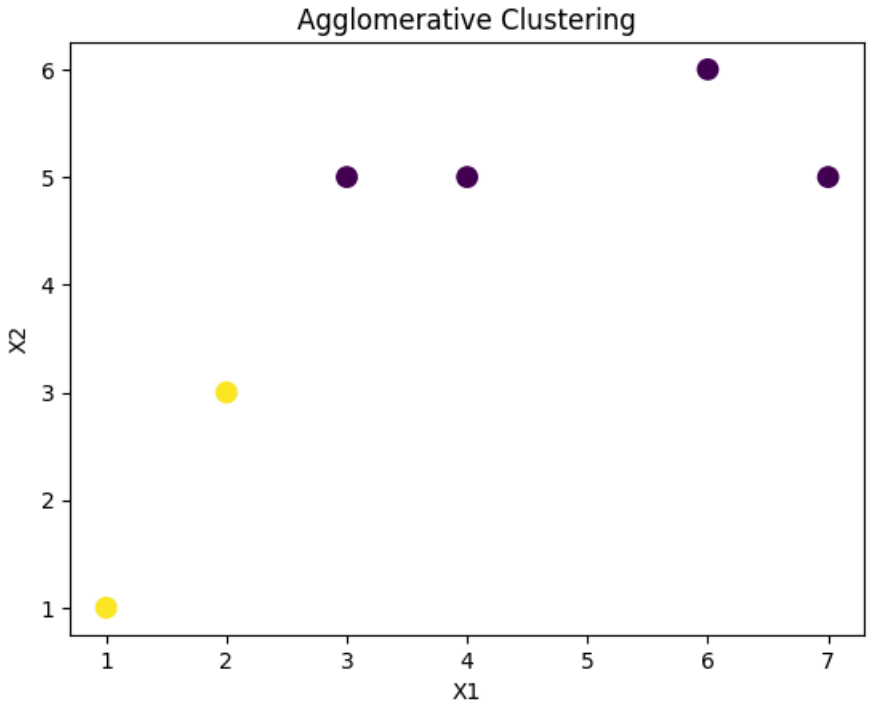
```
# Chuyển thành định dạng DataFrame
X_df_average = pd.DataFrame(X, columns=['X1', 'X2'])

# Gọi hàm để phân loại
model_average = AgglomerativeClustering(
    n_clusters=2,        # Số lượng nhóm
    linkage='average',    # Cách tính khoảng cách tâm cụm
    metric='euclidean'   # Cách tính khoảng cách giữa các điểm
)

X_df_average['cluster'] = model_average.fit_predict(X_df_average)
X_df_average
```

Out[13]:

	X1	X2	cluster
0	1	1	1
1	2	3	1
2	3	5	0
3	4	5	0
4	6	6	0
5	7	5	0

In [14]:

```
plt.scatter(
    X_df_average['X1'],
    X_df_average['X2'],
    c=X_df_average['cluster'],
    s=80
)

plt.xlabel('X1')
plt.ylabel('X2')
plt.title('Agglomerative Clustering')
plt.show()
```



2. Phân cụm theo thứ bậc với dữ liệu lớn.

File dữ liệu **oceanic_basalts.csv** chứa thông tin của nghiên cứu về thành phần khoáng chất có trong trầm tích đáy đại dương. Các mẫu được thu thập từ nhiều khu vực đại dương trên toàn thế giới. Dữ liệu bao gồm các biến

- affinity: loại trầm tích basalts, bao gồm 3 loại: island arc basalts (IAB - bazan vòng cung đảo); mid oceanic ridge basalts (MORB - bazan sống núi giữa đại dương); ocean island basalts (OIB - bazan đảo đại dương)
- SiO2: hàm lượng % SiO2 trong mẫu
- TiO2: hàm lượng % TiO2 trong mẫu
- Al2O3: hàm lượng % Al2O3 trong mẫu
- CaO: hàm lượng % CaO trong mẫu
- MgO: hàm lượng % MgO trong mẫu
- MnO: hàm lượng % MnO trong mẫu
- K2O: hàm lượng % K2O trong mẫu
- Na2O: hàm lượng % Na2O trong mẫu

In [15]:
```
# Đọc dữ liệu
url = '/content/drive/MyDrive/course_Data_Science_for_Geoscience/Week 1/Dataset/oceanic_basalts.csv'
df = pd.read_csv(url)
```

In [16]:
```
df.head(5)
```

Out[16]:

	affinity	SiO2	TiO2	Al2O3	CaO	MgO	MnO	K2O	Na2O
0	OIB	46.10	2.38	14.24	10.45	9.25	0.15	0.63	3.10
1	OIB	45.68	2.87	13.09	10.12	10.84	0.18	1.13	3.38
2	OIB	45.07	3.29	12.17	10.82	11.40	0.18	1.28	3.18
3	OIB	45.00	3.70	12.10	10.80	8.59	0.17	1.04	2.62
4	OIB	45.60	3.68	10.70	11.90	10.20	0.19	0.99	2.14

In [17]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 646 entries, 0 to 645
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   affinity  646 non-null    object
 1   SiO2      646 non-null    float64
 2   TiO2      646 non-null    float64
 3   Al2O3     646 non-null    float64
 4   CaO       646 non-null    float64
 5   MgO       646 non-null    float64
 6   MnO       646 non-null    float64
 7   K2O       646 non-null    float64
 8   Na2O      646 non-null    float64
dtypes: float64(8), object(1)
memory usage: 45.6+ KB
```

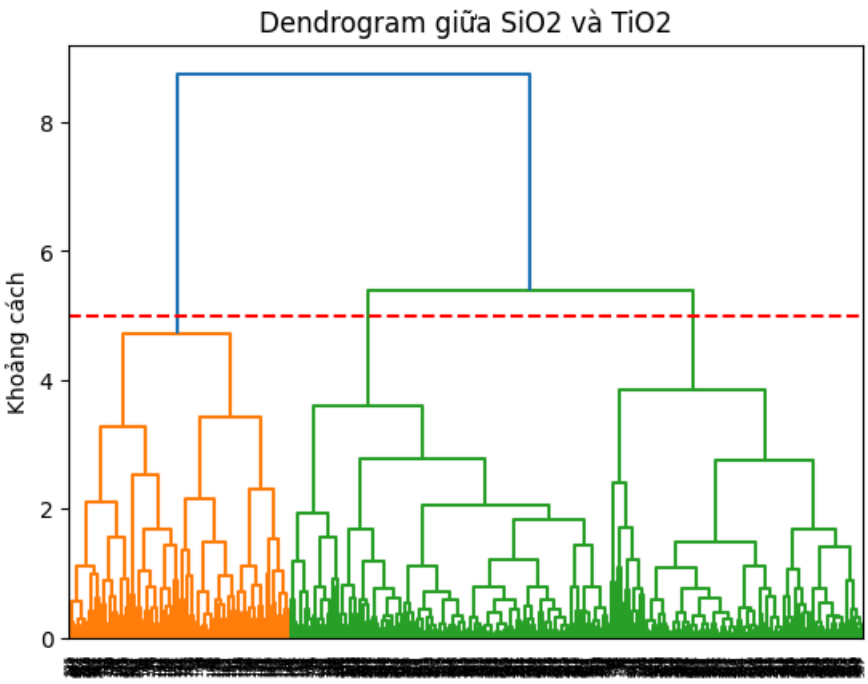
In [18]:
```
df = df[['SiO2', 'TiO2', 'Al2O3', 'CaO', 'MgO', 'K2O', 'Na2O']]
```

Và để trực quan các điểm dữ liệu theo các nhóm theo số cụm, chúng ta sử dụng hàm `AgglomerativeClustering` trong thư viện `sklearn.cluster`

Với dữ liệu nhiều chiều, ta cần lựa chọn là số cột hợp lý (hoặc dùng các phương pháp giảm số chiều) để giảm chiều dữ liệu, từ đó giúp việc phân cụm tốt hơn. Như ví dụ ở phía dưới, ta chọn 2 cột dữ liệu là `SiO2` và `TiO2` với phương pháp là `complete` sẽ có được Dendrogram như hình.

In [19]:
```
X_2D = df[['SiO2', 'TiO2']]
```

In [20]:
```
Z = shc.linkage(X_2D, method='complete', metric='euclidean')

shc.dendrogram(Z)
plt.ylabel("Khoảng cách")
plt.title("Dendrogram giữa SiO2 và TiO2")
plt.axhline(y=5, color='r', linestyle='--')
plt.show()
```



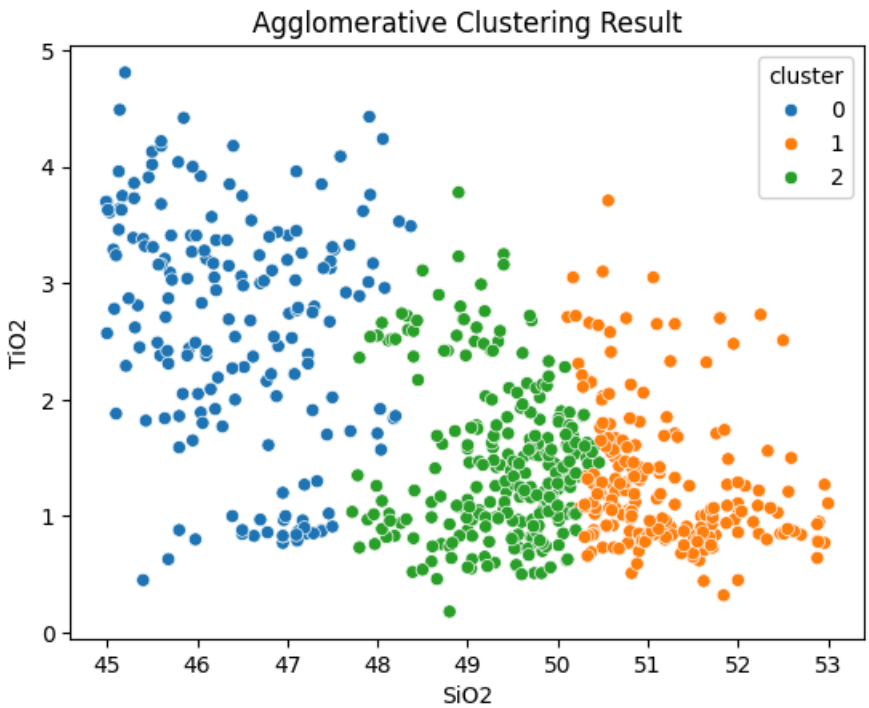
Tương tự với mảng dữ liệu ở mục **1**, ta định nghĩa mô hình phân cụm và lựa chọn số cụm hợp lý thông qua Dendrogram ở phía trên. Ở đây, nếu ta đang chấp nhận sự khác biệt tối đa giữa các cụm là 5 đơn vị, thì sẽ phân thành được 3 cụm dữ liệu.

In [21]:
```
from sklearn.cluster import AgglomerativeClustering

df_cluster_2d = X_2D.copy()
```

In [22]:
```
# Clustering
model = AgglomerativeClustering(
    n_clusters=3,
    linkage='complete',
    metric='euclidean'
)
df_cluster_2d['cluster'] = model.fit_predict(X_2D)
```

Cuối cùng, là biểu diễn thông tin dữ liệu với các cụm dữ liệu sau khi phân chia bằng biểu đồ phân tán.

In [23]:
```
sns.scatterplot(
    x = 'SiO2',
    y = 'TiO2',
    data = df_cluster_2d,
    hue = 'cluster',
    palette = 'tab10'
)

plt.title('Agglomerative Clustering Result')
plt.show()
```



Để làm rõ hơn về sự khác biệt giữa các nhóm, ta có thể sử dụng hàm `groupby` theo từng cụm để thống kê cơ bản giữa các nhóm.

In [24]:
```
df_cluster_2d.groupby('cluster').mean()
```

Out[24]:

cluster	SiO2	TiO2
0	46.443333	2.636811
1	51.171561	1.284141
2	49.378751	1.459483

Tương tự với việc lấy 2 cột, ví dụ tiếp theo sẽ lấy thông tin của 3 cột là **SiO2**, **TiO2** và **Al2O3**, việc phân cụm sẽ tương tự với các ví dụ phía trên.

In [25]:
```
X_3D = df[['SiO2', 'TiO2', 'Al2O3']]
```

In [26]:
```
X_3D
```

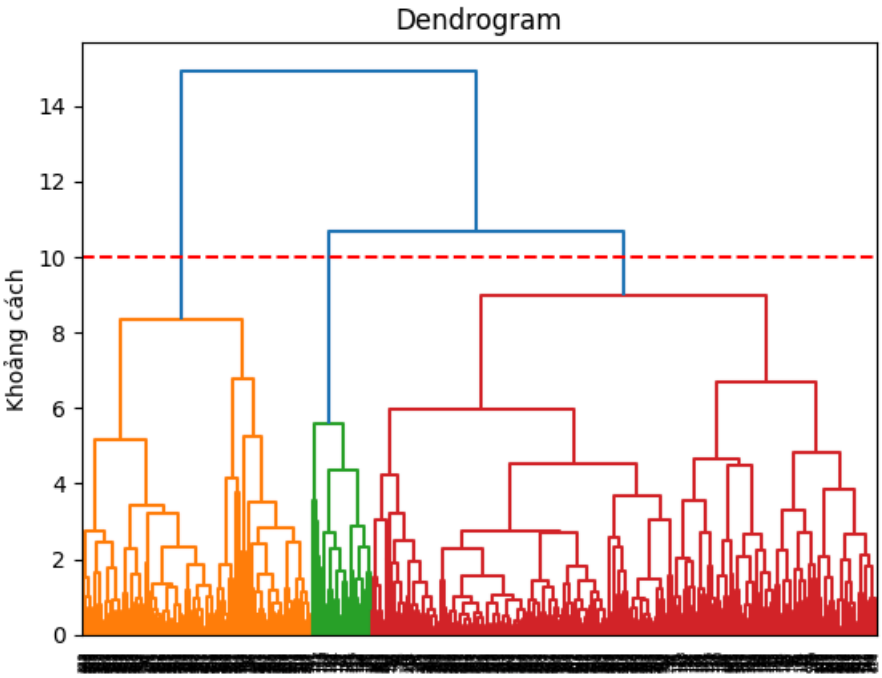
Out[26]:

	SiO2	TiO2	Al2O3
0	46.10	2.38	14.24
1	45.68	2.87	13.09
2	45.07	3.29	12.17
3	45.00	3.70	12.10
4	45.60	3.68	10.70
...	...	...	...
641	51.84	0.32	16.79
642	50.82	1.00	17.50
643	48.88	1.09	17.75
644	52.36	1.07	19.33
645	48.83	0.77	18.85

646 rows × 3 columns

Ở đây, ví dụ sử dụng phương pháp **complete**, có thể thay thế tham số `linkage` bằng các phương pháp khác như `single` hoặc `average`.

In [27]:
```
Z = shc.linkage(X_3D, method='complete', metric='euclidean')

shc.dendrogram(Z)
plt.axhline(y=10, color='r', linestyle='--')
plt.ylabel("Khoảng cách")
plt.title("Dendrogram")
plt.show()
```

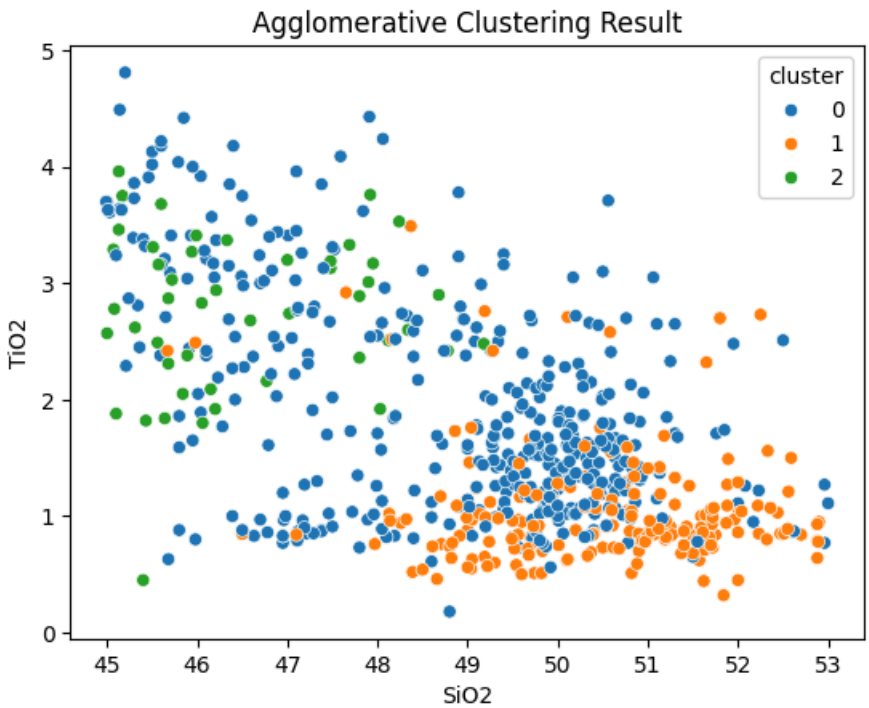


Tiếp theo cần định nghĩa mô hình phân cụm và lựa chọn số cụm hợp lý thông qua Dendrogram ở phía trên. Ở đây, nếu ta đang chấp nhận sự khác biệt tối đa giữa các cụm là 10 đơn vị, thì sẽ phân thành được 3 cụm dữ liệu.

In [28]:
```
from sklearn.cluster import AgglomerativeClustering

df_cluster_3d = X_3D.copy()

# Clustering
model = AgglomerativeClustering(
    n_clusters=3,
    linkage='complete',
    metric='euclidean'
)

df_cluster_3d['cluster'] = model.fit_predict(X_3D)
```

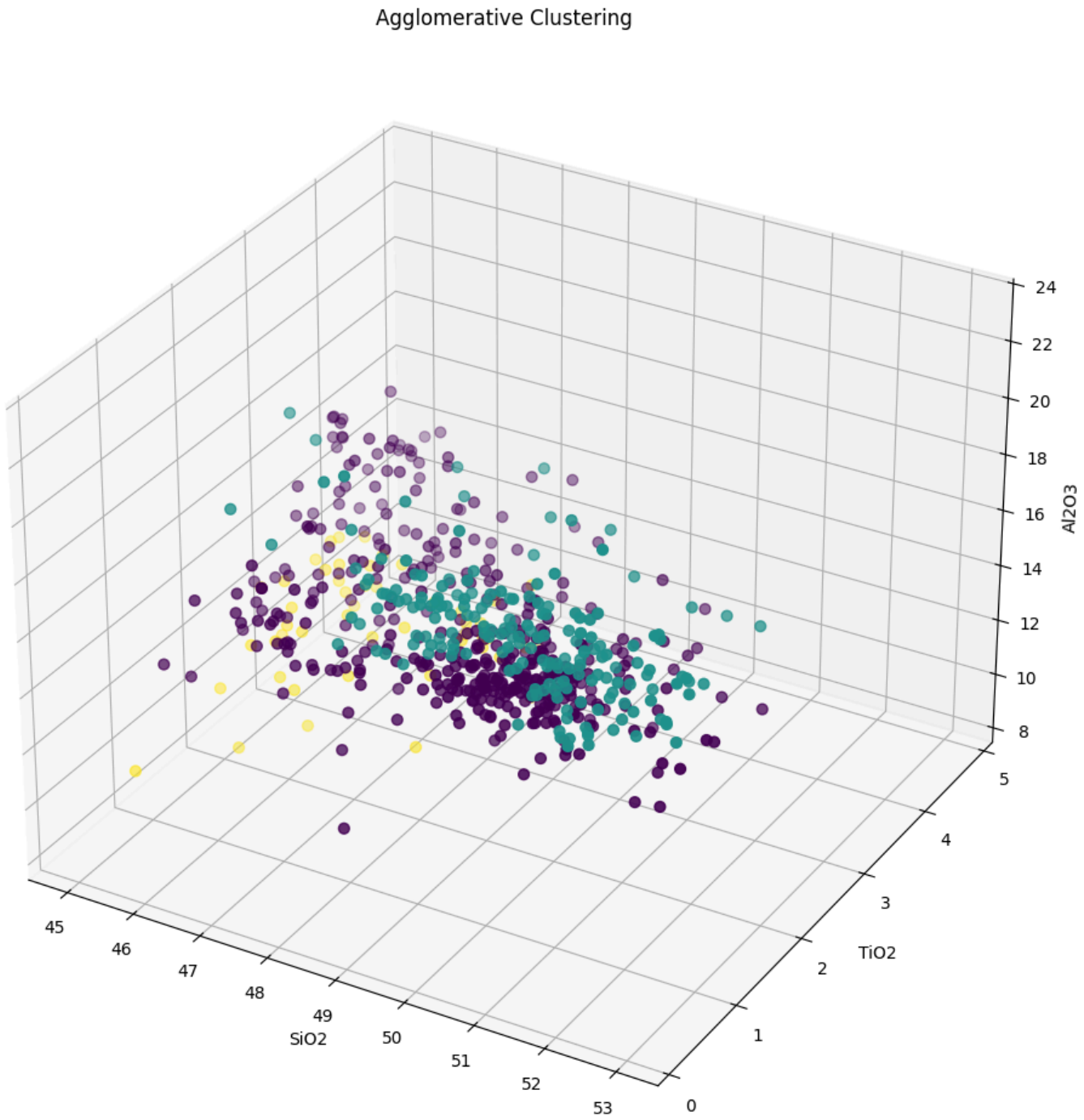
Cuối cùng là biểu diễn các điểm dữ liệu trên biểu đồ phân tán.

In [29]:
```
sns.scatterplot(
    x = 'SiO2',
    y = 'TiO2',
    data = df_cluster_3d,
    hue = 'cluster',
    palette = 'tab10'
)

plt.title('Agglomerative Clustering Result')
plt.show()
```



Có thể thấy, vì biểu đồ phân tán 2 chiều ta chỉ có thể chọn 2 biến đại diện cho trục x và trục y, vì vậy không thể đảm bảo trực quan với các điểm dữ liệu 3 biến. Vì vậy, cần biểu diễn chúng trên hệ trục 3 chiều Oxyz như phía dưới.

In [30]:
```
fig = plt.figure(figsize=(14, 12))    # Định nghĩa khung
ax = fig.add_subplot(projection='3d') # Biểu diễn 3 chiều

scatter = ax.scatter(
    df_cluster_3d['SiO2'],
    df_cluster_3d['TiO2'],
    df_cluster_3d['Al2O3'],
    c=df_cluster_3d['cluster'],
    s=40
)

ax.set_xlabel('SiO2')
ax.set_ylabel('TiO2')
ax.set_zlabel('Al2O3')
ax.set_title('Agglomerative Clustering')

plt.show()
```



3. Phương pháp K-means

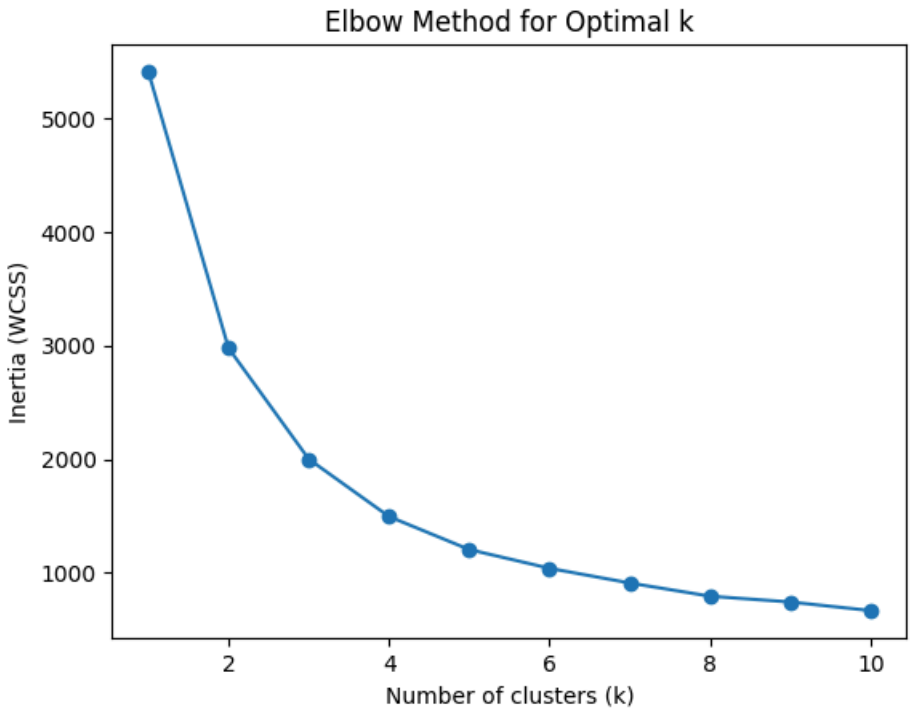
Với phương pháp phân cụm theo thứ bậc (Hierarchical clustering), bước đầu tiên chúng ta đang xem xét mỗi điểm dữ liệu sẽ là một cụm, sau đó sẽ gom các điểm dữ liệu lại thành nhóm mới dựa trên khoảng cách. Còn với phương pháp phân cụm K-means dữ liệu ban đầu được xem là một cụm tổng thể, sau đó chọn số lượng cụm ngẫu nhiên, từ đó chia các điểm dữ liệu theo số lượng cụm theo các bước:

- **Bước 1**: Lựa chọn K, là số nhóm mà chúng ta muốn phân cụm dữ liệu.
- **Bước 2**: Chọn ngẫu nhiên K đối tượng để làm tâm của K cụm ban đầu.
- **Bước 3**: Gán từng đối tượng cho nhóm có tâm gần nhất với đối tượng đó.
- **Bước 4**: Tính lại K tâm của các cụm mới, là trung bình cộng của các điểm trong cụm dữ liệu.
- **Bước 5**: Lặp lại từ bước 3 cho đến khi tâm không còn di chuyển nữa.

Trong thực tế, việc chọn số cụm K là không đơn giản, mặc dù ta thường có xu hướng chọn K = 2 , 3 hoặc 4, nhưng có thể có những trường hợp cho số cụm nhiều hơn 4. Do đó, để xác định K phù hợp nhất với dữ liệu, ta thường sử dụng biểu đồ khuỷu tay (Elbow plot), trong đó, giá trị V 1 , V 2 , . . . , V K (tức là tổng bình phương trong cụm) được biểu diễn theo chiều tăng của số cụm, từ 1 tới K . Khi tăng số cụm K , đường cong giảm mạnh lúc đầu rồi sau đó giảm chậm lại. Điểm “gãy” (giống khuỷu tay) là lựa chọn tốt cho K .

Như ví dụ phía dưới, ta cần chạy vòng lặp với số K cụm từ 1 đến thứ K, chúng ta tính toán tổng bình phương trong cụm để xác định số cụm tối ưu thông qua biểu đồ khuỷu tay với 2 cột dữ liệu là `SiO2` và `Al2O3`.

In [31]:
```
from sklearn.cluster import KMeans

X_kmeans = df[['SiO2', 'Al2O3']]

inertias = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(
        n_clusters=k,
        random_state=42
    )
    kmeans.fit(X_kmeans)
    inertias.append(kmeans.inertia_)

# Plot Elbow
plt.plot(K_range, inertias, marker='o')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method for Optimal k')
plt.show()
```



Có thể thấy qua biểu đồ khuỷu tay, ở số cụm K = 3 là điểm hợp lý để chia dữ liệu. Tiếp theo, ta cần định nghĩa mô hình K-Means với số cụm đã xác định.

In [32]:
```
kmeans = KMeans(
    n_clusters=3,
    random_state=42
)
```

Với hàm `fit_predict` với đầu vào là dữ liệu ban đầu sẽ in ra "nhãn" cụm với từng điểm dữ liệu thuộc về.

In [33]:
```
labels = kmeans.fit_predict(X_kmeans)
labels
```

Out[33]:
```
array([2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2,
       1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 0, 2, 2, 0, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 2, 1, 1, 2, 2, 2,
       2, 2, 2, 2, 2, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 0, 2,
       2, 2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 1, 2, 2, 2,
       2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 2, 2, 2, 1, 2, 2,
       1, 2, 2, 1, 1, 2, 0, 0, 2, 2, 2, 1, 2, 2, 1, 0, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0,
       1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 2, 2,
       2, 2, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1,
       0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 2, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 0, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 2, 1, 0, 1,
       0, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 2, 0, 0, 0, 1, 0, 0, 0,
       1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 2, 0, 2, 0, 1, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 2, 2, 0, 0, 2, 2, 1, 2, 0, 2,
       2, 2, 2, 2, 0, 2, 2, 1, 0, 0, 2, 0, 0, 2, 0, 2, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 2, 0, 2, 0, 0, 0, 0, 0, 2, 0, 0, 0,
       0, 0, 0, 1, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1,
       1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1,
       1, 1, 0, 0, 2, 1, 0, 0, 0, 0, 1, 0, 2, 1, 0, 0, 0, 2, 2, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0], dtype=int32)
```

Và với hàm `kmeans.cluster_centers_` sẽ in ra giá trị tâm cụm sau quá trình tính toán.

In [34]:
```
centroids = kmeans.cluster_centers_
centroids
```

Out[34]:
```
array([[50.17514035, 18.29409357],
       [50.17763158, 15.16098246],
       [46.61710526, 14.10689474]])
```

Tiếp theo, chúng ta sẽ thêm cột `cluster` nhằm lưu trữ thông tin cụm của từng điểm dữ liệu.

In [35]:
```
X_kmeans.loc[:, 'cluster'] = labels
```

```
/tmp/ipython-input-3160383388.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  X_kmeans.loc[:, 'cluster'] = labels
```

In [36]:
```
X_kmeans
```

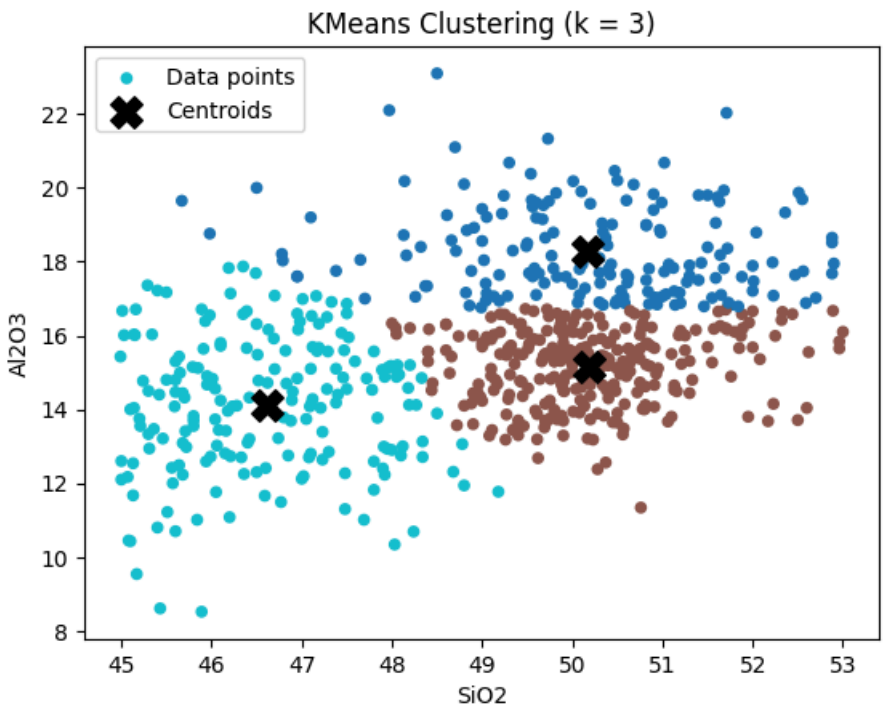
Out[36]:

	SiO2	Al2O3	cluster
0	46.10	14.24	2
1	45.68	13.09	2
2	45.07	12.17	2
3	45.00	12.10	2
4	45.60	10.70	2
...	...	...	...
641	51.84	16.79	0
642	50.82	17.50	0
643	48.88	17.75	0
644	52.36	19.33	0
645	48.83	18.85	0

646 rows × 3 columns

Cuối cùng là vẽ biểu đồ phân tán các điểm dữ liệu, cùng với tâm cụm.

In [37]:
```
plt.scatter(
    X_kmeans['SiO2'],
    X_kmeans['Al2O3'],
    c=labels,
    cmap='tab10',
    s=20,
    label='Data points'
)

# Plot centroids
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    c='black',
    s=200,
    marker='X',
    label='Centroids'
)

plt.xlabel('SiO2')
plt.ylabel('Al2O3')
plt.title('KMeans Clustering (k = 3)')
plt.legend()
plt.show()
```



3. Phương pháp phân cụm K-means kết hợp với PCA.

Như đã giới thiệu trong các phần trước, dữ liệu địa chất thường có nhiều biến, và các biến có đơn vị cũng như khoảng giá trị xác định khác nhau. Để khắc phục nhược điểm này ta có thể kết hợp phân tích thành phần chính (PCA) và thuật toán phân cụm. Trong đó, PCA được thực hiện ở bước đầu nhằm giúp giảm số chiều của dữ liệu mà vẫn giữ lại cấu trúc chính (sự biến động là tương đương), làm dữ liệu "gọn" hơn. Sau đó, các thuật toán phân cụm được áp dụng trên dự liệu mới, có số chiều nhỏ hơn, các biến có cùng khoảng giá trị xác định và độc lập.

Quy trình kết hợp PCA và thuật toán phân cụm được trình bày như sau:

- Bước 1. Chuẩn hoá dữ liệu.
- Bước 2. Áp dụng PCA cho dữ liệu đã chuẩn hóa. Sau đó, giữ lại từ 2 tới 4 thành phần chính mà giải thích phần lớn sự biến động của dữ liệu gốc (50% tới 80%).
- Bước 3. Tiến hành phân cụm trên dữ liệu của các thành phần chính được xác định trong Bước 2.
- Bước 4. Hiển thị kết quả trên biểu đồ biplot, trong đó, các điểm dữ liệu được tô màu theo cụm đã được xác định trong Bước 3, và các loading vector được xác định trong Bước 2. Điều này cho phép ta nhìn thấy mỗi cụm khác nhau ở hướng nào, biến nào quyết định sự khác biệt giữa các cụm.

Bước 1: Như đã đề cập ở phía trên, đầu tiên chúng ta cần chuẩn hóa dữ liệu với thư viện `StandardScaler`

In [38]:
```
from sklearn.preprocessing import StandardScaler
```

In [39]:
```
# --- Lấy dữ liệu số ---
X = df.select_dtypes(include='number')
```

In [40]:
```
X
```

Out[40]:

	SiO2	TiO2	Al2O3	CaO	MgO	K2O	Na2O
0	46.10	2.38	14.24	10.45	9.25	0.63	3.10
1	45.68	2.87	13.09	10.12	10.84	1.13	3.38
2	45.07	3.29	12.17	10.82	11.40	1.28	3.18
3	45.00	3.70	12.10	10.80	8.59	1.04	2.62
4	45.60	3.68	10.70	11.90	10.20	0.99	2.14
...	...	...	...	...	...	...	...
641	51.84	0.32	16.79	12.16	6.79	0.16	1.19
642	50.82	1.00	17.50	10.28	7.82	0.39	2.61
643	48.88	1.09	17.75	11.92	7.37	0.60	2.09
644	52.36	1.07	19.33	10.14	3.46	0.54	2.38
645	48.83	0.77	18.85	12.59	5.71	0.60	2.07

646 rows × 7 columns

In [41]:
```
# --- Chuẩn hóa ---
X_scaled = StandardScaler().fit_transform(X)
X_scaled
```

Out[41]:
```
array([[-1.53104363,  0.68372451, -0.68150882, ...,  0.64687181,
         0.01018576,  0.62846842],
       [-1.74328441,  1.20064951, -1.22565235, ...,  1.29085331,
         0.92331417,  1.02986872],
       [-2.05153889,  1.64372808, -1.66096717, ...,  1.5176644 ,
         1.19725269,  0.74315422],
       ...,
       [-0.12621177, -0.67715966,  0.97931186, ..., -0.11456543,
        -0.04460194, -0.81943981],
       [ 1.63235473, -0.69825864,  1.72691775, ..., -1.69819289,
        -0.15417735, -0.40370378],
       [-0.15147853, -1.01474333,  1.49979697, ..., -0.78689832,
        -0.04460194, -0.84811126]])
```

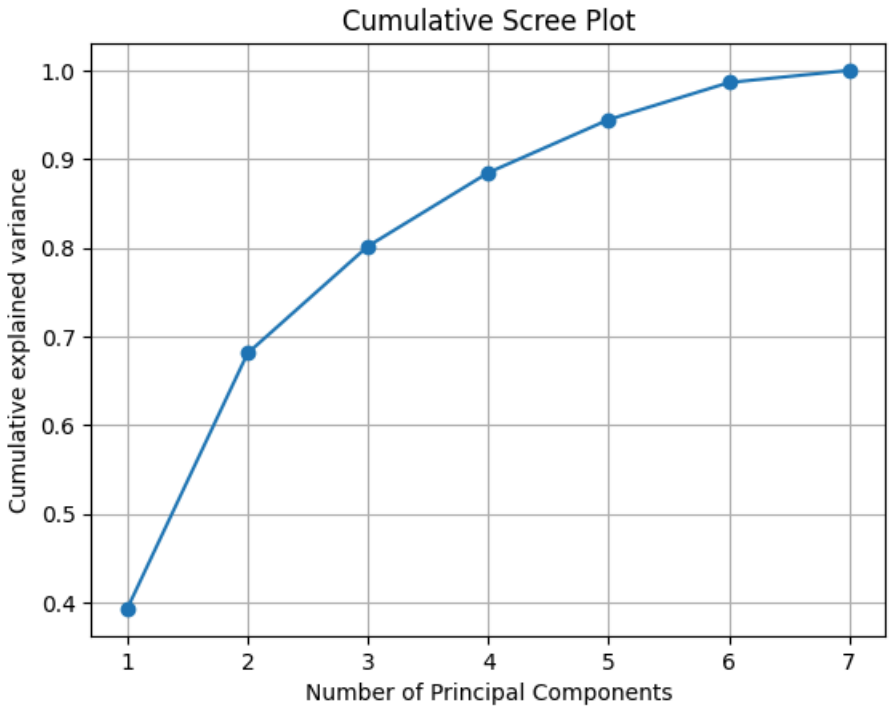
Bước 2: Áp dụng PCA cho dữ liệu đã chuẩn hóa. Sau đó, giữ lại từ 2 tới 4 thành phần chính mà giải thích phần lớn sự biến động của dữ liệu gốc

In [42]:
```
from sklearn.decomposition import PCA
pca = PCA()
pca.fit(X_scaled)
```

Out[42]:
```
▾ PCA
PCA()
```

In [43]:
```
explained = pca.explained_variance_ratio_
cumulative = []
total = 0

for val in explained:
    total += val
    cumulative.append(total)

cumulative = np.array(cumulative)
```

In [44]:
```
plt.plot(
    range(1, len(cumulative) + 1),
    cumulative,
    marker='o'
)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative explained variance")
plt.title("Cumulative Scree Plot")
plt.grid(True)
plt.show()
```



Với số thành phần chính "tối ưu" mà thể hiện qua biểu đồ khuỷu tay, ta có thể xác định số thành phần chính cần giữ lại là 2. Sau đó, cần chuyển đổi dữ liệu đã được chuẩn hóa theo số lượng thành phần chính.

In [45]:
```
# --- PCA ---
pca = PCA(n_components=2)
scores = pca.fit_transform(X_scaled)
scores
```

Out[45]:
```
array([[ 1.53842222,  0.91499383],
       [ 2.75482715,  1.21067141],
       [ 3.08155198,  1.85725232],
       ...,
       [-1.25343541, -0.01532956],
       [-1.72247758, -2.31059574],
       [-1.83002133, -0.56786629]])
```

In [46]:
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Explained variance:", pca.explained_variance_ratio_.sum())
```

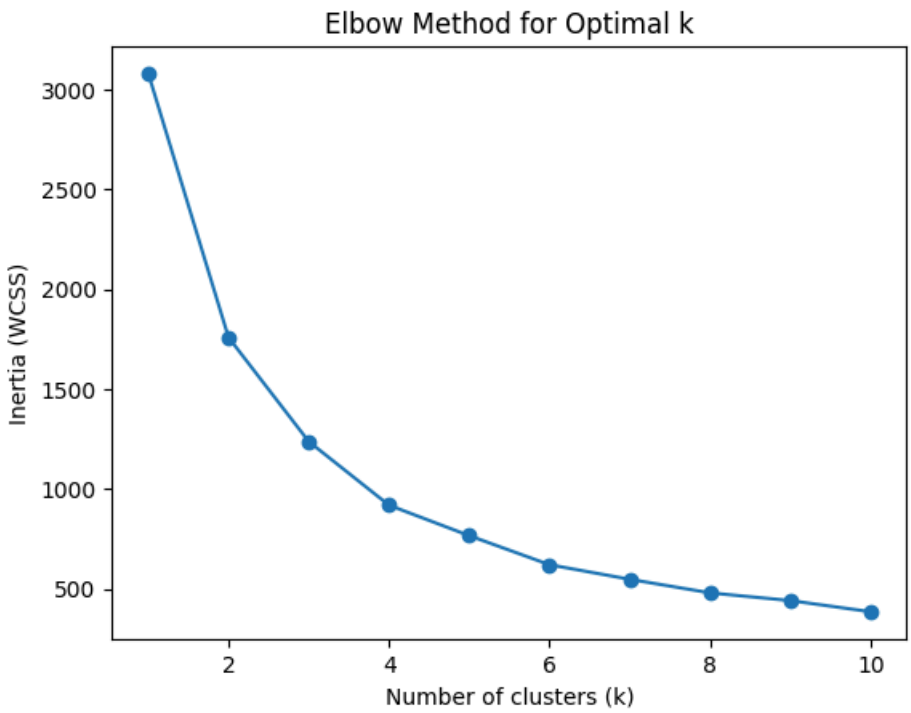
```
Explained variance: 0.68109301377729
```

Bước 3: Tiến hành phân cụm trên dữ liệu của các thành phần chính được xác định trong Bước 2.

Để phân cụm trên các thành phần chính, chúng ta cần biểu đồ khuỷu tay với tổng bình phương trong cụm với số lượng cụm K.

In [47]:
```
from sklearn.cluster import KMeans

inertias = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(
        n_clusters=k,
        random_state=42
    )
```

```
    kmeans.fit(X_pca) # Áp dụng trên dữ liệu thành phần chính
    inertias.append(kmeans.inertia_)

# Plot Elbow
plt.plot(K_range, inertias, marker='o')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method for Optimal k')
plt.show()
```



Sau khi chọn được số lượng cụm tối ưu là 3, ta định nghĩa mô hình K-means và áp dụng lên dữ liệu thành phần chính

In [48]:
```
from sklearn.cluster import KMeans

kmeans = KMeans(
    n_clusters=3,
    random_state=42
)

labels = kmeans.fit_predict(X_pca)
centroids = kmeans.cluster_centers_
```

In [49]:
```
# --- Gộp lại với dữ liệu ban đầu ---
df_pca = df.copy()
df_pca['PC1'] = X_pca[:, 0]
df_pca['PC2'] = X_pca[:, 1]
df_pca['cluster'] = labels
df_pca
```

Out[49]:

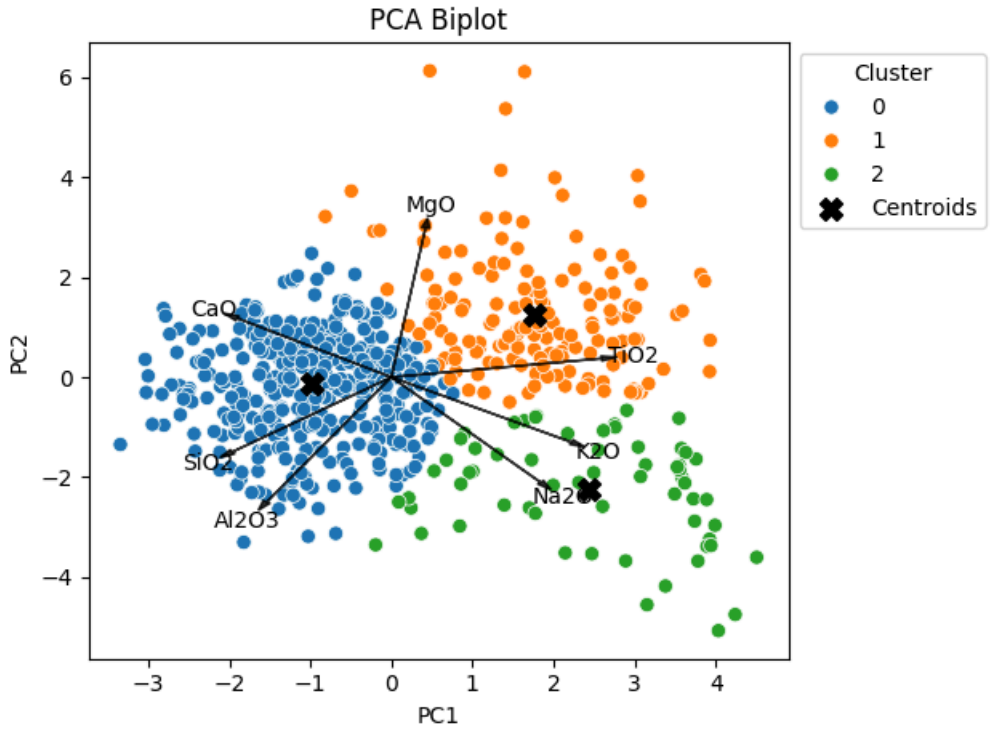
	SiO2	TiO2	Al2O3	CaO	MgO	K2O	Na2O	PC1	PC2	cluster
0	46.10	2.38	14.24	10.45	9.25	0.63	3.10	1.538422	0.914994	1
1	45.68	2.87	13.09	10.12	10.84	1.13	3.38	2.754827	1.210671	1
2	45.07	3.29	12.17	10.82	11.40	1.28	3.18	3.081552	1.857252	1
3	45.00	3.70	12.10	10.80	8.59	1.04	2.62	2.749347	1.685068	1
4	45.60	3.68	10.70	11.90	10.20	0.99	2.14	2.281774	2.813737	1
...	...	...	...	...	...	...	...	...	...	...
641	51.84	0.32	16.79	12.16	6.79	0.16	1.19	-3.037857	0.356267	0
642	50.82	1.00	17.50	10.28	7.82	0.39	2.61	-1.079655	-0.640822	0
643	48.88	1.09	17.75	11.92	7.37	0.60	2.09	-1.253435	-0.015330	0
644	52.36	1.07	19.33	10.14	3.46	0.54	2.38	-1.722478	-2.310596	0
645	48.83	0.77	18.85	12.59	5.71	0.60	2.07	-1.830021	-0.567866	0

646 rows × 10 columns

Bước 5: Hiển thị kết quả trên biểu đồ biplot, trong đó, các điểm dữ liệu được tô màu theo cụm đã được xác định trong Bước 3, và các loading vector được xác định trong Bước 2

In [50]:

```
# --- Hệ số phóng đại độ dài vector ---
arrow_scale = 5
loadings = pca.components_.T * arrow_scale

# --- Vẽ scatter plot (phân loại theo loại đá) ---
sns.scatterplot(
    data=df_pca,
    x='PC1',
    y='PC2',
    hue='cluster',
    s=45,
    palette = 'tab10'
    #alpha=0.6,
    #edgecolor='black'
)

# Plot centroids
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    c='black',
    s=100,
    marker='X',
    label='Centroids'
)

# --- Vẽ vector tải (không enumerate) ---
for (x, y), feature in zip(loadings, X.columns):
    plt.arrow(0, 0, x, y,
              color='black', alpha=0.8,
              head_width=0.1, head_length=0.2, linewidth=1)

    plt.text(x*1.15, y*1.15, feature,
             color='black', fontsize=10,
             ha='center', va='center')


# --- Hoàn thiện ---
plt.xlabel(f"PC1")
plt.ylabel(f"PC2")
plt.title("PCA Biplot")
plt.legend(title='Cluster', bbox_to_anchor=(1, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



Kết quả cho thấy các mẫu địa chất trong cụm số 2 có xu hướng chứa nhiều SiO2 , Al2O3 và Na2O, tuy nhiên, Al2O3 dường như không có tương quan với Na2O (hai vector loading gần như vuông góc), trong khi đó, SiO2 và Al2O3 là có tương quan tuyến tính thuận với nhau.